

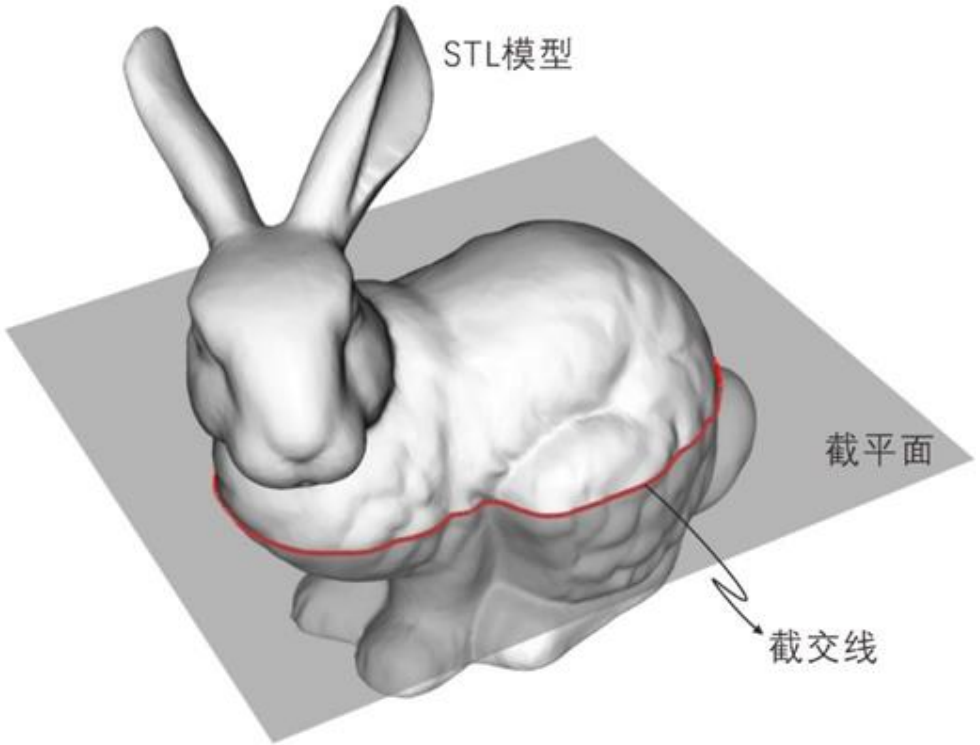
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

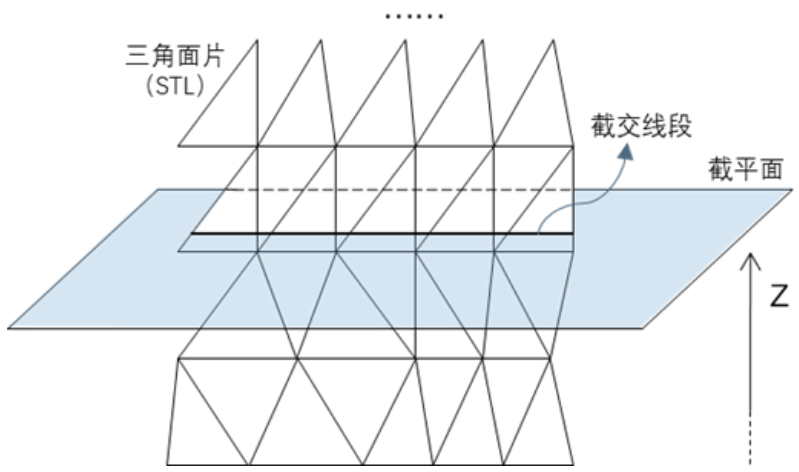
第5章 STL模型截交计算优化

- 暴力截交存在的问题
- 扫描平面法截交优化
- 层高匹配法截交优化
- 算法测试比较



1. 暴力截交存在的问题

- **STL模型截交的本质**：用一系列层高已知的平行平面和STL模型**求交**，计算并保存得到的截交线段（**散乱无序**）以备后面使用。
- **暴力截交效率低下**。在使用暴力截交算法时，对每张截平面，都会遍历整个STL模型中的所有三角面片，调用平面和三角面片的基础求交函数（GeomAlgo模块中的**intersectTriangleZPlane**）计算截交线段。
  - 如果STL模型中有100万个三角面片，那么一张截平面一次截交就要调用100万次求交函数。
  - 为了加深直观感受，用之前写的Python代码进行了实际测试，测试用主机CPU型号为intel Core i7-8700 @ 3.20GHz，内存大小为24GB。对于一个拥有约45000个三角面片的STL模型，在切片厚度为0.1mm，切片层数为2202层的情况下，使用暴力法消耗的截交运算总时间约为107秒（约2分钟）。在三维打印中，拥有4万多面片的STL模型不算大，对于面片规模更大的模型，使用暴力法无疑将消耗更多的时间。
- **暴力截交效率低的原因：使用暴力法截交，大部分求交计算可能都是无用（冤枉，invalid）的。**
  - 对于某一指定高度的截平面，它和模型中多数三角面片的相对位置决定了它们之间根本就不可能相交，如图所示，最低一排和最高一排的三角面片就不可能和截平面相交。
  - 虽然在intersectTriangleZPlane函数中做了一些非常重要的预处理，即如果截平面高于三角面片最高点或低于三角面片最低点，则直接返回None，事实也的确证明这些预处理能够降低不少求交计算量（如果注释掉intersectTriangleZPlane函数中的预处理代码，在相同的测试环境和对象下，截交运算时间约为1444秒，即约24分钟），但**仍然不能规避每张截平面和模型中所有面片遍历求交的弊病**。

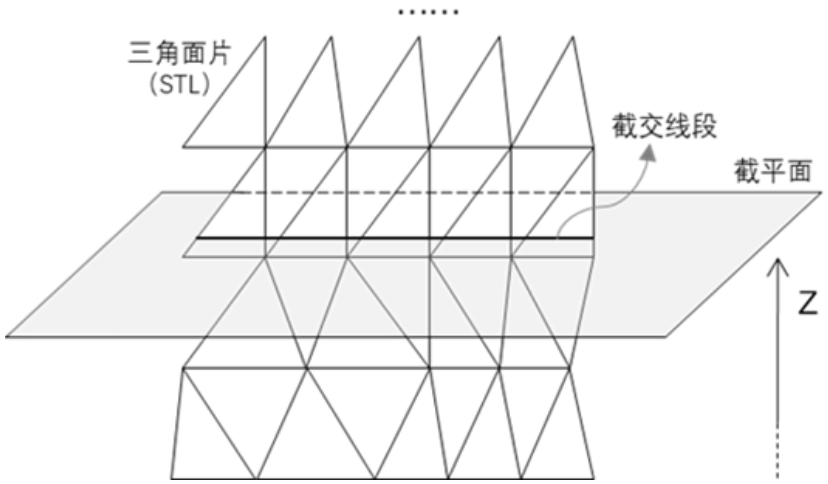


截平面和模型中少部分三角面片相交

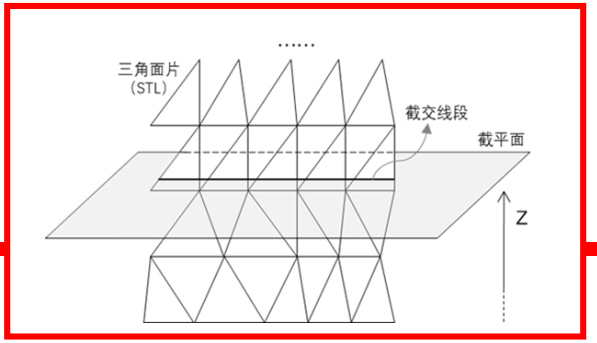
2. 扫描平面法截交优化

• 基本思路

- 对一张高度从低到高连续抬升的截平面和一个STL模型中的所有三角面片，如果能够**事先预知哪些面片处在截平面附近（可能相交），哪些面片离截平面较远（不可能相交），**那么就可以将截平面附近的面片**筛选**出来，并在当前层上，仅计算截平面和这些附近面片的交线，从而实现减少求交次数的目标。
- 这样似乎又回到了求交函数**intersectTriangleZPlane**中的预处理筛选过程，但其实这是两个层次的事情，这里说的减少求交次数是在求交函数之外减少求交次数。
- 那么问题是**如何预知那些面片位于截平面附近**呢？这里介绍使用**扫描平面（sweep plane）**法快速获取截平面附近的三角面片。



2. 扫描平面法截交优化



• 扫描平面数据结构

- **概念**：扫描平面和截平面一样，也是平行于坐标系XY平面的平面，其高度（z值）可按需要沿坐标系Z轴从低到高（或从高到低）连续变化，扫描截交STL模型。
- **特性**：**扫描平面维护了一个与其相交的三角面片列表**，即模型中所有最高点高于（或等于）和最低点低于（或等于）扫描平面的三角面片将被存入这个列表。当扫描平面在Z轴上移动时，其三角面片列表中的面片将**动态增减**。
- **优势**：STL模型截交问题可以转化为扫描平面和其自身三角面片列表中保存的面片求交的问题，这样就可以避免遍历STL模型中的所有面片。
- 据此，创建一个扫描平面类，类名为**SweepPlane**，把这个类写在IntersectStl_sweep.py文件中，IntersectStl_sweep模块专门用于封装基于扫描平面法的STL模型截交计算代码。

IntersectStl_sweep.py	扫描平面类，扫描平面法中的关键数据结构	5-1

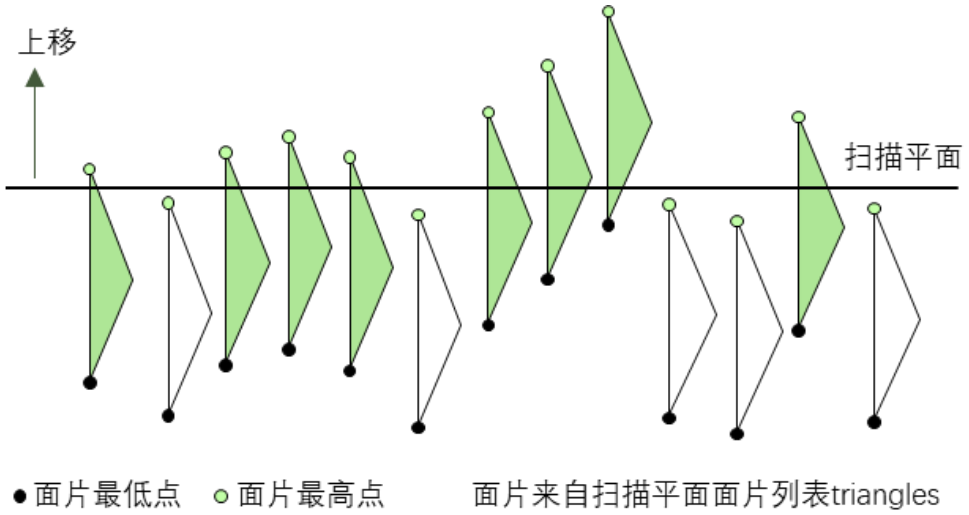
```
1  ……                                  # 导入必要模块
2  class SweepPlane:
3      def __init__(self):
4          self.triangles = []         # 三角面片列表
5      pass
```

- SweepPlane类非常简单，仅有一个列表类型的属性，即triangles，该列表用于维护和扫描平面相交的三角面片数组。还可以往SweepPlane类中添加表示平面当前高度的属性z，但这并不是必要的。

2. 扫描平面法截交优化

• 扫描平面法原理

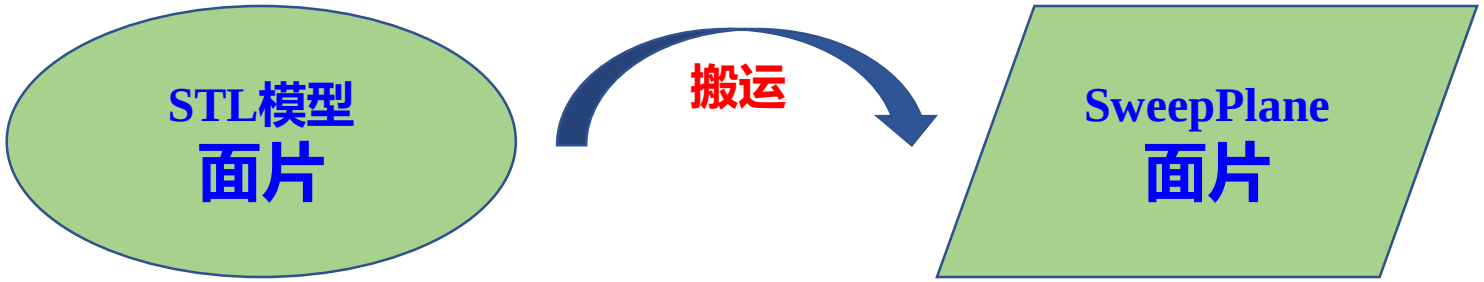
- 结合SweepPlane数据结构，当扫描平面从模型最低点往上向模型最高点连续扫描过程中，扫描平面中的三角面片列表triangles将不断**更新**，即：
  - **老的面片可能被不断移除**
  - **新的面片又可能被不停地添加进来**
- **移除老的面片**。判断扫描平面的triangles中哪些面片需要被移除相对简单。因为triangles中维护的面片数量相对整个STL模型拥有的面片数量来说较少，因此每次只需遍历triangles中所有面片，如果某个面片的最低点低于或最高点高于扫描平面高度，那么就需要将该面片从triangles中移除。



2. 扫描平面法截交优化

- **扫描平面法原理：添加新的面片**

- 如果采用暴力算法，只需遍历STL模型中的所有面片，判断哪些面片同时满足最高点高于（或等于）扫描平面高度且最低点低于（或等于）扫描平面高度，那么就将该面片添加进triangles。如果真的这样做，那么基于扫描平面的截交算法相对于之前介绍的暴力截交法将不会有任何运算效率的提升。



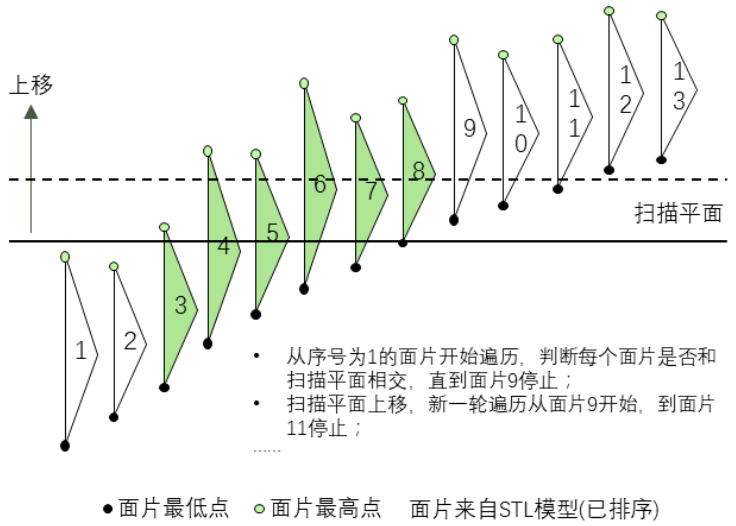
如果STL模型中的三角面片不是杂乱无序的，而是按照**从低到高**的顺序排好序的，那么当扫描平面在向上扫描时，以上向triangles中添加新面片的过程是否减少一些遍历的运算量？

2. 扫描平面法截交优化

- **扫描平面法原理：添加新的面片**

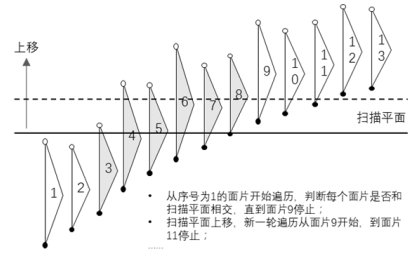
- 顺着以上思路，如果模型中的每个三角面片是按照**最低点的z坐标从小到大排序**的，那么当扫描平面上移时，只需按模型中的**面片序号从小到大依次遍历**判断：
  - 如果当前序号对应的三角面片最高点高于（或等于）扫描平面高度且最低点低于（或等于）扫描平面高度，那么就将该平面添加至扫描平面的triangles中；
  - 一旦出现当前序号对应的面片的最低点高于扫描平面高度，则记录该面片序号（不妨记为k，图中为面片9）并立即停止遍历流程。
  - 此时求取并保存扫描平面和triangles中每个三角面片的截交线段。
  - 然后扫描平面继续向上平移，重复上述步骤，遍历模型中三角面片的起始序号变为上次记录的序号k。

- **排序提升添加片面速度的原因**：如果STL模型中某个序号的面片在扫描平面以上（即该面片最低点高于扫描平面），那么**所有大于这个序号的面片必然都在扫描平面以上。**



从已排序的STL模型面片列表向扫描平面添加面片示意

2. 扫描平面法截交优化



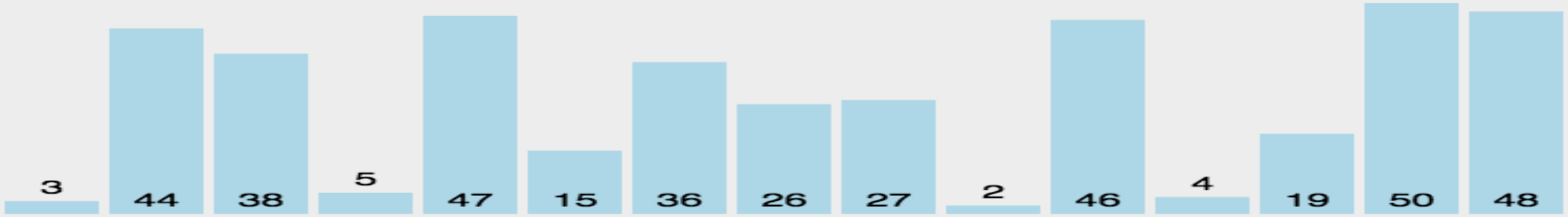
- **三角面片排序**

- **排序问题**是计算机算法领域最为经典的问题之一，解决排序问题有众多算法，如常见的冒泡排序（bubble sort）、插入排序（insertion sort）、归并排序（merge sort）、快速排序（quick sort）等等，目前较快的排序算法可以做到O(nlog(n))乃至O(n)的时间复杂度，其中n为排序个数。

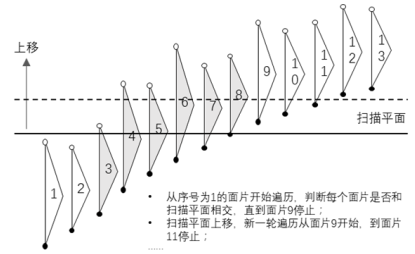
- 几乎在任何编程语言的标准库里，都内置的排序算法，如C语言中的qsort，C++ std库中的sort，C#以及java中的sort，Python也不例外，它也有内置sort函数，可直接对**列表对象（StlModel中的triangles）**进行排序。

可以直接对triangles调用sort函数吗?
即：triangles.sort()

十大经典排序算法（动图演示）：https://www.cnblogs.com/onepixel/articles/7674659.html



2. 扫描平面法截交优化



• 三角面片排序

- **两个三角形如何比较大小？**结合应用背景定义一种三角形**大小比较规则**，对任意两个三角形T_1和T_2：
  - 如果T_1.zMinPnt().z < T_2.zMinPnt().z，则$T_1 < T_2$
  - 如果T_1.zMinPnt().z = T_2.zMinPnt().z，则$T_1 = T_2$
  - 如果T_1.zMinPnt().z > T_2.zMinPnt().z，则$T_1 > T_2$
- 在C、C++、C#等语言中，碰到这种情况时一般会为待比较的对象定义一个**比较回调函数**，作为参数输入sort函数中。
- 在Python中，只要针对sort的key参数指定比较指标即可，因此正确的排序写法是：

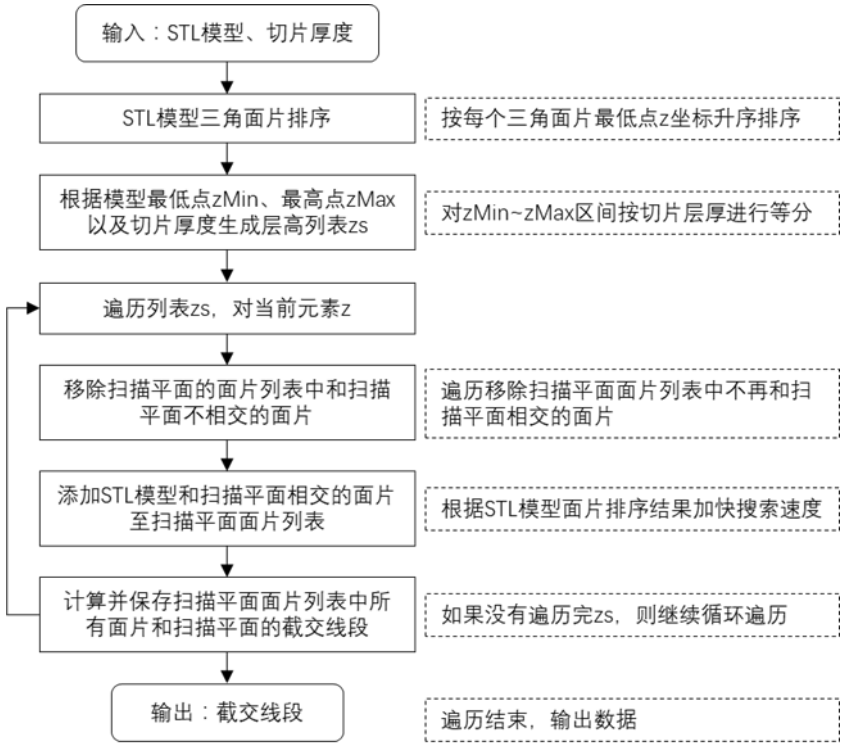
```
1 | triangles.sort(key = lambda t : t.zMinPnt().z, reverse = False)
```

- 在以上代码采用了Python的lambda表达式（即函数式表达式），首先使用zMinPnt函数获取三角形对象的最低点（参见Triangle类），然后在从该最低点获取z坐标作为比较指标。
- 默认情况下，Python的sort以**升序排序**，即reverse参数默认值为False；如果需要按降序排序，只需指定reverse参数值为True即可。

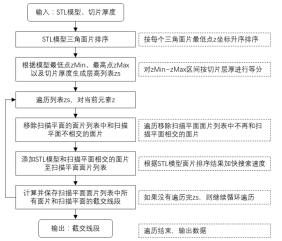
2. 扫描平面法截交优化

• 扫描平面法算法流程

- 流程输入为STL模型和切片厚度两个参数，输出为一系列截交线段。其中截交线段将以层的形式组织，存储在Layer对象的列表中。
- 循环的结束条件：层高列表zs中所有层高被遍历。



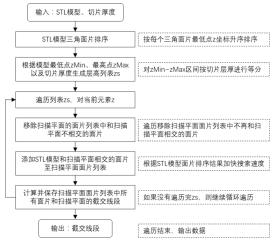
2. 扫描平面法截交优化



- **扫描平面法算法实现代码**

尝试自己实现下哈！

2. 扫描平面法截交优化



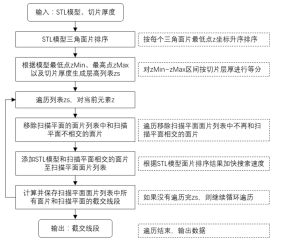
- **扫描平面算法函数解析**

- **__init__**函数输入STL模型和切片厚度参数，并在初始化函数中调用intersect核心函数完成模型和扫描平面截交计算功能，最终将得到的截交线段保存在列表layers中。

- **intersect**函数按照算法流程实现。需要说明的是：
  - 在移除扫描平面面片列表循环（5-2代码15~17行）中，使用了降序的方式遍历移除sweep.triangles中的三角面片，即从序号最大的面片开始向序号最小（0）的面片遍历，这并非是一时兴起的写法，而是有意为之的。为什么不从小到大移除?
  - 该函数有进一步优化的空间？如何优化?

- **genLayerHeights**函数的功能是根据切片厚度参数生成并返回具有均匀层高特性的层高列表。
  - 之所以将这个功能从intersect函数中单独拎出来，是考虑到后期假如有非均匀层高切片的需求，只需修改这个函数的实现代码即可，其接口可以不用改变。

2. 扫描平面法截交优化



• 扫描平面算法使用

- 以上实现了基于扫面平面法的STL模型截交算法，可以参照下列语句使用该算法：

```
1    layers = IntersectStl_sweep(stlModel, layerThk).layers
```

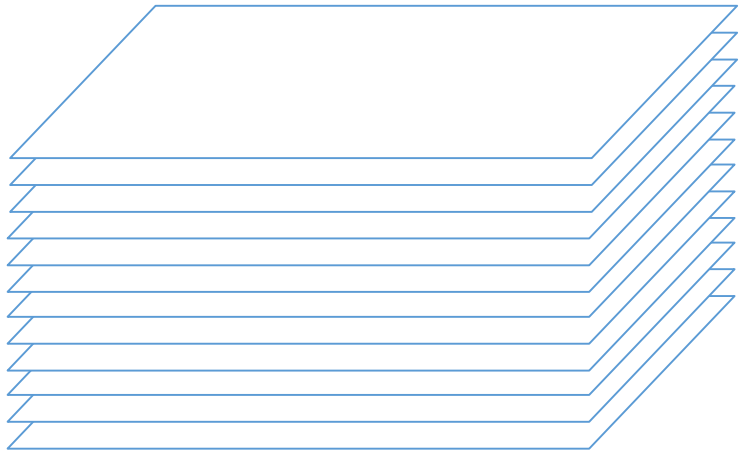
- 其中stlModel为事先读取的StlModel对象，layerThk为事先指定的层高，layers用于保存计算得到的截交线段。
- 在SliceAlgo.py中添加一个全局函数，函数名为**intersectStl_sweep**，呼应intersectStl_brutal函数。该函数实现代码如下：

SliceAlgo.py | **intersectStl_sweep：添加扫描平面法STL模型截交函数** | **5-3**

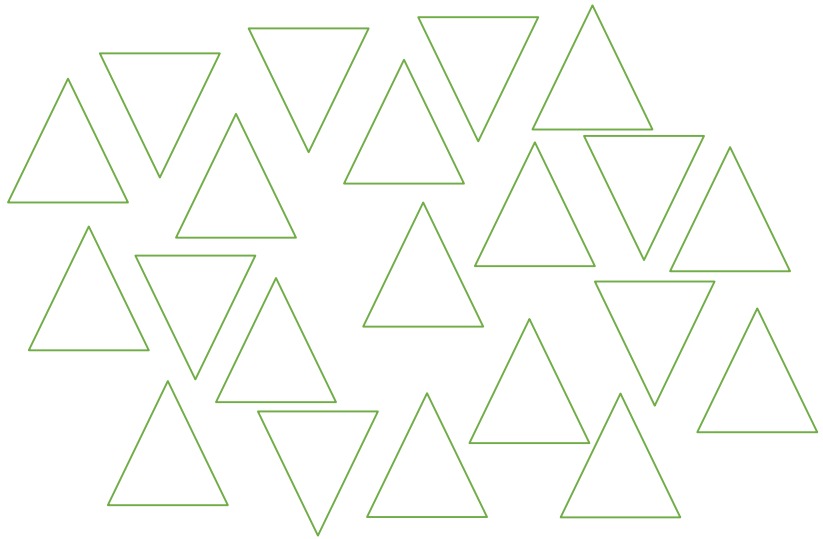
```
1  from IntersectStl_sweep import *          # 导入IntersectStl_sweep模块
2  ……                                        # 忽略SliceAlgo其他导入模块及代码
3  def intersectStl_sweep(stlModel, layerThk):      # 定义函数
4      return IntersectStl_sweep(stlModel, layerThk).layers    # 返回layers
```

3. 层高匹配法截交优化

- **切片需要解决的问题**：使用一组平行平面去截交一组三角面片，得到各层上的截交线段集。
- **一种思维定式**：无论是扫描平面法，还是暴力截交法，其本质思想都是尝试用截平面和STL模型中的所有三角面片求交，即求交时，截平面为**主角**，保持固定不变，三角面片为**配角**，不断被遍历更替。
- **是否可以尝试转变一下思维？**即：从STL模型中的三角面片出发，以面片为主角，截平面为配角，每次求交时，保持面片固定不变，不断遍历更替截平面。



平面集

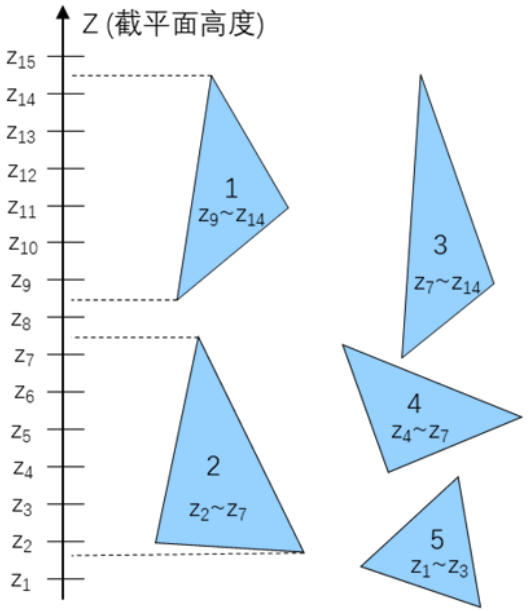


面片集

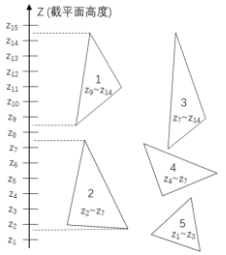
3. 层高匹配法截交优化

• 层高匹配法原理

- **逆向思维**：假设事先已经生成切片层高列表（zs），从STL模型中的三角面片出发，对每一个三角面片，遍历所有层高上的截平面，计算并保存截交线段。
  - **问题**：从全局的角度考虑，假设当前STL模型中的三角面片数量为N，层高列表中层高数为M，那么从三角面片角度出发求交，所需的三角形和平面求交总次数为N　M，这和从截平面角度出发的暴力截交法求交总次数无异（M　N）。**这样做好像没什么意义！**
- 在这种思维模式下，如何降低求交次数？
- **突破口：**层高列表**zs是一组已经从小到大排好顺序的实数**，这组实数代表了截平面在坐标系Z轴上的高度。对一个给定的三角面片，可以非常容易地把它在Z方向上包含的层高数值筛选出来。
  - 对1号三角面片，它包含的层高为z_9~z_{14}；
  - 对2号三角面片，它包含的层高为z_2~z_7；
  - 对3号三角面片，它包含的层高为z_7~z_{14}……
  - 从三角面片的角度出发，在对1号面片求交时，只需求依次求取该面片和高度为z_9~z_{14}的6个截平面的交线即可，而无需遍历所有层高上的截平面，从而节省大量不必要的运算。



3. 层高匹配法截交优化



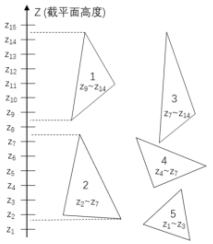
- **层高匹配法原理**

Triangle.py	三角形类，封装了3个顶点坐标和1个法向量坐标	4-1

```
1   from GeomBase import *
2   class Triangle:
3       def __init__(self, A, B, C, N = Vector3D(0,0,0)):
4           self.A, self.B, self.C, self.N = A.clone(), B.clone(), C.clone(), N.clone()      # 顶点和法向
5           self.zs = []                                                                      # 存储面片包含的层高
6       def __str__(self):
7           pass
8       def zMinPnt(self):                                                                    # 3个顶点中的Z方向最低点
9           pass
10      def zMaxPnt(self):                                                                    # 3个顶点中的Z方向最高点
11          pass
```

- **数据结构支持**：在Triangle类中，预留了一个未曾使用的zs属性，这个属性正是为保存三角面片包含的层高数值用的。
- **子任务**：从层高列表zs中为STL模型中的每个三角面片匹配层高。

3. 层高匹配法截交优化



- **层高匹配法原理**

- 创建IntersectStl_match类，在类中封装**matchFacetZs_brutal**函数（暴力匹配），实现面片层高匹配功能。

IntersectStl_match.py	matchFacetZs_brutal：将层高匹配到每个三角面片，暴力法	5-4

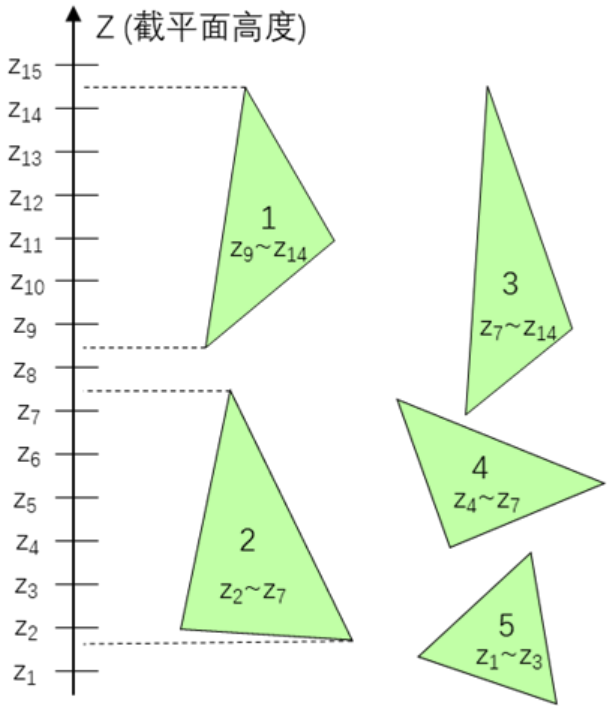
```
1   ……                                          # 导入必要模块
2   class IntersectStl_match:
3       def __init__(self, stlModel, layerThk):  # 初始化，输入模型和层高
4           self.stlModel = stlModel             # 保存模型和层高
5           self.layerThk = layerThk
6           self.layers = []                     # 切片结果，Layer对象列表
7           self.intersect()                     # 调用截交核心函数
8       def intersect(self): pass                # 截交核心函数定义，略
9       def matchFacetZs_brutal(self, zs):       # 暴力法层高匹配函数
10          for tri in self.stlModel.triangles:
11              zMin, zMax = tri.zMinPnt().z, tri.zMaxPnt().z  # 面片最高点最低点
12              for z in zs:
13                  if z > zMax: break
14                  if z >= zMin and z <= zMax:  # 如果层高在面片最低最高点
15                      tri.zs.append(z)         # 之间，则将其加入面片
```

3. 层高匹配法截交优化

• 层高匹配法原理

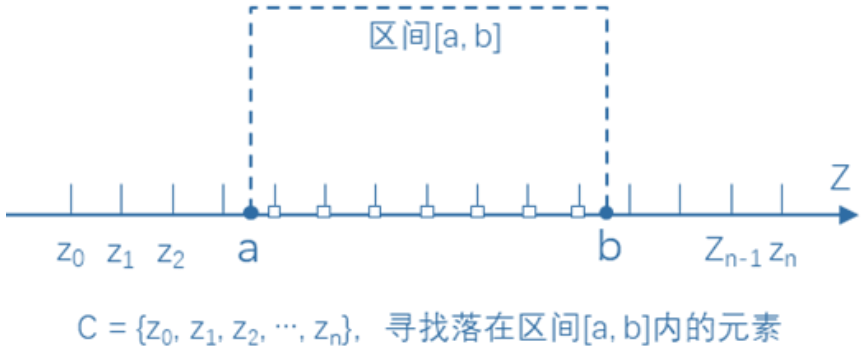
- **matchFacetZs_brutal**函数输入参数为层高列表zs，注意，zs在输入前必须已按照升序排序。对STL模型中的每个三角面片
  - 首先计算面片最低点和最高点的z坐标，即zMin和zMax；
  - 然后从头开始遍历zs中每个元素（记为z）：如果该$z \le zMax$且$z \ge zMin$，则将z添加至模型面片zs属性中；一旦出现$z > zMax$的情况，则退出当前面片的遍历过程。
  - 之所以可以在$z > zMax$时退出遍历循环，是因为层高列表zs已经过升序排序，这样可以节省一些遍历比较时间。

- **matchFacetZs_brutal不足之处**：**上限易找，下限难求**
  - 如图中的1号三角面片，用以上方法匹配层高时，处在z_{15}以上的层高比较过程确实可以被规避，但z_9以下的层高只能逐个试探，而事实上只有z_9~z_{14}的层高对1号面片来说是有用的。
  - 层高下限（z_9）的寻找只能从层高列表的开头开始遍历，这可能会浪费一定的时间，特别是对于相对位置较高的三角面片；而一旦找到了层高下限，层高上限（z_{14}）就比较容易确定了。



3. 层高匹配法截交优化

- **二分法层高匹配**
- **层高匹配问题的抽象描述**：对序列C = {z_0, z_1, z_2, …, z_n}，满足0 ≤ i < j ≤ n，则z_i ≤ z_j，在序列中寻找落在区间[a, b]中的元素。



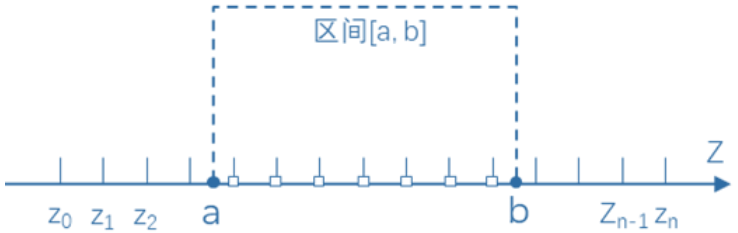
C = {z_0, z_1, z_2, ⋯, z_n}，寻找落在区间[a, b]内的元素

- 如果C元素个数足够少，那么可以逐个遍历zs，找出落在[a，b]区间内的元素。但这里认为C包含足够多的元素，而且算法的执行时间是主要考量的指标。
- C中的元素满足单调递增的特性，因此**只需在序列中找出落在[a, b]区间内的元素上、下限对应的序号即可**。

3. 层高匹配法截交优化

• 二分法层高匹配：下限寻找

- **目标**：**在C中搜索和a最接近的元素。**
- 在二分法中，设3个变量：low、up、mid，在本例中，这3个变量为非负整数，表示C中**元素的序号**。二分法搜索下限具体步骤如下：
  - 步骤1. 初始化，令low = 0，up = n。
  - 步骤2. 计算mid值，mid = (low + up) / 2，注意这里除法为整数除法。
  - 步骤3. 如果$z_{mid} < a$，则low = mid，否则up = mid。
  - 步骤4. 如果up - low ≤ 1，则退出循环，否则跳至步骤2。
- **收敛条件**：二分法搜索循环在运行过程中，low和up值不断更新，最终如果它们之间差值不大于1，则退出循环，算法收敛。
- 此时a必然落在z_{low}和z_{up}内，即：$z_{low} \le a \le z_{up}$，则z_{up}必然包含在区间[a, b]内。但判断z_{low}是否在区间[a, b]内需要进一步看不等式$z_{low} \le a$中的等号是否能够取到。
- **下限**：如果$z_{low} = a$，那么C中落在区间[a, b]的元素下限为z_{low}，否则下限为z_{up}。

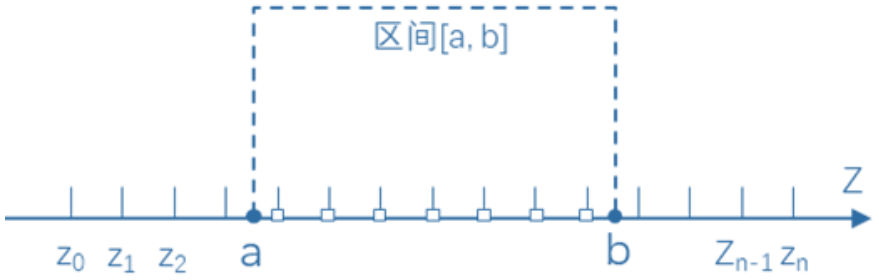


C = {$z_0, z_1, z_2, \cdots, z_n$}，寻找落在区间[a, b]内的元素

3. 层高匹配法截交优化

• 二分法层高匹配：上限寻找

- **目标**：在C中搜索和b最接近的元素。
- 在二分法中，设3个变量：low、up、mid，在本例中，这3个变量为非负整数，表示C中**元素的序号**。二分法搜索下限具体步骤如下：
  - 步骤1. 初始化，令low = 0，up = n。
  - 步骤2. 计算mid值，mid = (low + up) / 2，注意这里除法为整数除法。
  - 步骤3. 如果$z_{mid} < b$，则low = mid，否则up = mid。
  - 步骤4. 如果up - low ≤ 1，则退出循环，否则跳至步骤2。
- **收敛条件**：二分法搜索循环在运行过程中，low和up值不断更新，最终如果它们之间差值不大于1，则退出循环，算法收敛。
- b必然落在z_{low}和z_{up}内，即：$z_{low} \le b \le z_{up}$，此时z_{low}必然包含在区间[a, b]内。但判断z_{up}是否在区间[a, b]内需要进一步看不等式$b \le z_{up}$中的等号是否能够取到。
- **上限**：如果$z_{up} = b$，那么C中落在区间[a, b]的元素上限为z_{up}，否则上限为z_{low}。



C = {$z_0, z_1, z_2, \cdots, z_n$}，寻找落在区间[a, b]内的元素

3. 层高匹配法截交优化

• 二分法层高匹配实现

- 面给出二分法层高匹配代码。把这部分代码写在IntersectStl_match类中，创建一个**matchFacetZs_bisection**函数，代码如下：

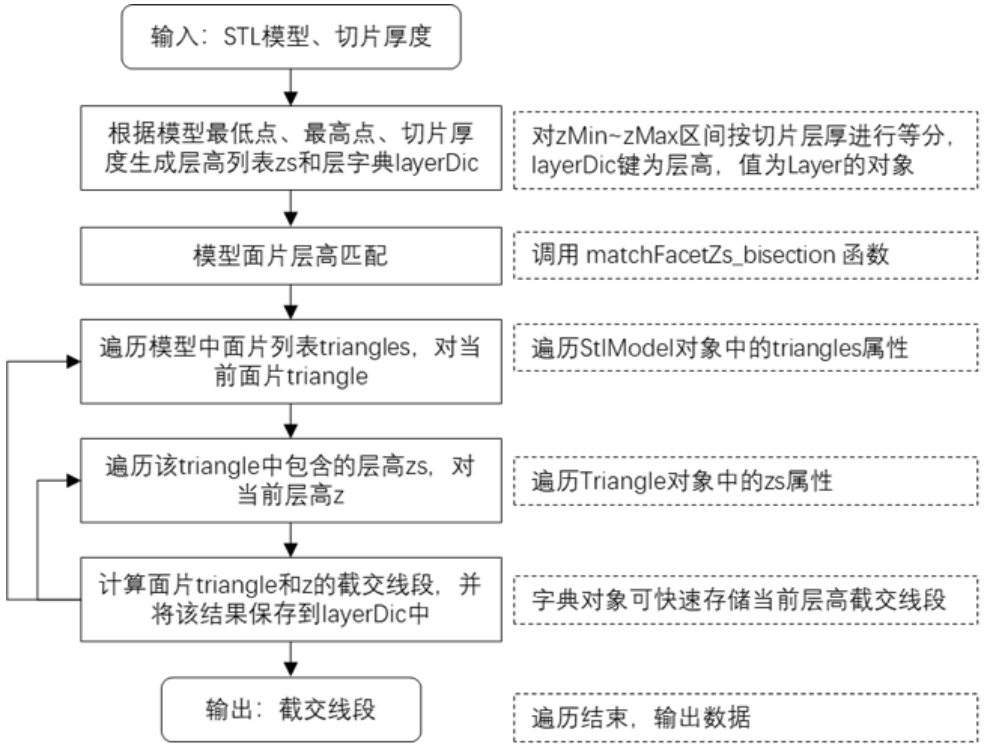
IntersectStl_match.py	matchFacetZs_ bisection：将层高匹配到每个三角面片，二分法	5-5

```
1   ……                                          # 导入必要模块
2   class IntersectStl_match:
3       ……                                      # 省略前面代码
4       def matchFacetZs_bisection(self, zs):   # 二分法层高匹配函数
5           n = len(zs)                         # 层高列表元素个数
6           for tri in self.stlModel.triangles: # 遍历模型中每个三角面片
7               zMin, zMax = tri.zMinPnt().z, tri.zMaxPnt().z   # 面片最低点、最高点
8               low, up, mid = 0, n - 1, 0      # 初始化low、up、mid
9               while up - low > 1:             # 1. 二分法求下限序号
10                  mid = int((low + up)/2)
11                  if zs[mid] < zMin: low = mid
12                  else: up = mid
13              start = up                      # 下限序号start
14              if mid == zMin:
15                  start = low
16              low, up = 0, n – 1              # 2. 二分法求上限序号
17              while up - low > 1:
18                  mid = int((low + up)/2)
19                  if zs[mid] < zMax: low = mid
20                  else: up = mid
21              stop = low                      # 上限序号stop
22              if mid == zMax:
23                  stop = up
24              for i in range(start, stop + 1, 1):   # 面片层高匹配[start, stop]
25                  tri.zs.append(zs[i])
```

3. 层高匹配法截交优化

• 层高匹配截交优化算法流程

- 流程输入为STL模型和切片厚度两个参数，输出为一系列截交线段。其中截交线段将以层的形式组织，存储在字典layerDic中。
- 循环的结束条件：STL模型中的所有三角面片被遍历。



3. 层高匹配法截交优化

- **层高匹配截交优化算法实现代码**

尝试自己实现下哈！

3.层高匹配法截交优化

- **层高匹配截交优化代码解析**

- **genLayerHeights**函数。和这个函数几乎一模一样的函数在IntersectStl_sweep类中已经实现了一遍，所不同的是，在这个函数中，除了返回层高列表之外，还返回了**层字典layerDic**。根据layerDic，只要指定层高z值，就能快速查询并获得处于该层高处的Layer对象，如layerDic[z_0]表示高度为z_0处的Layer对象。

- **intersect**函数中，只需4步，便能完成优化截交功能，即：层高生成、层高匹配、截交计算、数据转换。
  - 在截交计算这一步中，对当前面片triangle中的当前层高z，获得了截交线段seg并判断seg非空后，首先通过layerDic[z]检索到该层高下的层对象，然后将seg添加到该对象的segments列表中。这里必须要用字典才能完成快速检索的功能。如果不用层字典而用层列表，那么对每条截交线段，需要遍历层列表寻找相应高度的层对象，费时费力。
  - 函数返回时，将字典对象转化成了列表对象。

3. 层高匹配法截交优化

• 层高匹配截交优化算法使用

- 和IntersectStl_sweep类类似，可以参照下列语句使用该算法：

```
1    layers = IntersectStl_match(stlModel, layerThk).layers
```

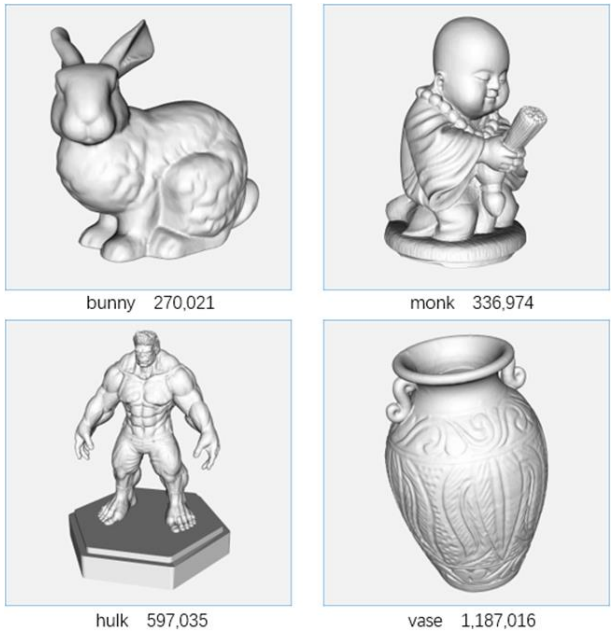
- 其中stlModel为事先读取的StlModel对象，layerThk为事先指定的层高，layers用于保存计算得到的截交线段。
- 最后，在SliceAlgo模块中在添加关于IntersectStl_match类的接口函数**intersectStl_match**，实现代码如下：

SliceAlgo.py	intersectStl_match：添加层高匹配法STL模型截交函数	5-7

```
1    from IntersectStl_match import *    # 导入IntersectStl_match模块
2    ……                                  # 忽略SliceAlgo其他导入模块及代码
3    def intersectStl_match(stlModel, layerThk):    # 定义函数
4        return IntersectStl_match(stlModel, layerThk).layers        # 返回layers
```

4. 算法比较测试

- **任务**：测试比较3种截交算法的执行时间
  - 暴力法
  - 扫描平面法
  - 层高匹配法
- 测试模型



- 选取了4个具有代表性的STL模型用作测试，分别为bunny、monk、hulk、vase。
- bunny模型又叫“斯坦福兔子（Stanford bunny）”，在计算机图形领域非常有名，经常被用来验证各种图形算法。

模型	面片数	模型高度 (mm)
bunny	270021	49.5
monk	336974	175.5
hulk	597035	119
vase	1187016	381.4

4. 算法比较测试

- **测试代码**

Test_intersectStl.py	测试3种模型截交法运算性能	5-8

```
1   ……                                          # 导入SliceAlgo、StlModel模块
2   stlModel = StlModel()
3   stlModel.readStlFile("E:\\STL\\bunny.STL")   # 从文件读入测试STL模型
4   print('face count:', stlModel.getFacetNumber())  # 打印模型面片数
5   layerThks = [1.0, 0.5, 0.2]                 # 设置3种切片厚度：1.0, 0.5, 0.2mm
6   for layerThk in layerThks:
7     print('layerThk:', layerThk)              # 打印当前切片厚度
8     start = time.clock()                      # 1. 扫描平面法
9     layers1 = intersectStl_sweep(stlModel, layerThk)    # 在算法开始前获取CPU时间,
10    end = time.clock()                            # 在算法结束后获取CPU时间
11    print("Sweep: %f CPU seconds" % (end - start))      # 打印扫描平面法执行时间
12    start = time.clock()                      # 2. 层高匹配法
13    layers2 = intersectStl_match(stlModel, layerThk)
14    end = time.clock()
15    print("Match: %f CPU seconds" % (end - start))      # 打印层高匹配法执行时间
16    start = time.clock()                      # 3. 暴力法
17    layers3= intersectStl_brutal(stlModel, layerThk)
18    end = time.clock()
19    print("Brutal: %f CPU seconds" % (end - start))      # 打印层高匹配法执行时间
20    print('layer count:', len(layers1), '\n\n')  # 打印切片层数
```

- **测试平台：CPU为intel Core i7-8700 @ 3.20GHz，24GB内存**

4. 算法比较测试

- **测试结果**

模型	面片数	模型高度 (mm)	切片厚度 (mm)	层数	sweep (秒)	match (秒)	brutal (秒)	比值B/S	比值 B/M
bunny	270021	49.5	1.0	49	2.5	2.8	16.6	6.6	5.9
			0.5	98	3.5	5.1	32.3	9.2	6.3
			0.2	247	7	7.7	79.7	11.4	10.4
monk	336974	175.5	1.0	175	5.5	6.4	67.5	12.3	10.5
			0.5	351	9.5	14.5	134.2	14.1	9.3
			0.2	877	21.4	22.2	337.3	15.8	15.2
hulk	597035	119	1.0	119	6.3	7.7	73.2	11.6	9.5
			0.5	238	10	11.5	150.1	15	13.1
			0.2	595	20.7	22.9	359	17.3	15.7
vase	1187016	381.4	1.0	384	22.4	25.9	408.5	18.2	15.8
			0.5	769	41.7	60	828.5	19.9	13.8
			0.2	1924	92.6	94.4	2034.4	22	21.6

- **结论：**
  - 首先，扫描平面法（sweep）和层高匹配法（match）相对于暴力法（brutal），其执行效率基本上会高一个数量级（5.9~22倍），这验证了sweep和match法的确实有效。
  - 其次，就sweep法和match法而言，两种方法在执行效率上相差无几，处在同一水平线上，但**sweep法稍胜一筹**，基本比match法效率高10%以上。
- **思考**：如果去掉intersectTriangleZPlane函数中的预处理代码，对两种优化算法的运行时间会有影响吗？

本章小结

- **这一章主要针对三维打印路径规划中的模型截交问题展开讨论。实现了两种优化截交算法，即扫描平面法和层高匹配法。**
  - 扫描平面法的核心是首先对模型中的三角面片进行排序，然后用一扫描平面从低到高收集与之相交的面片并进行截交计算。
  - 层高匹配法的本质是以模型中的三角面片为主导，收集与之相交的层高并进行截交计算。
- **最后，对3种截交算法进行了测试比较，结果表明：扫描平面法和层高匹配法的性能持平，前者略胜一筹，但都比暴力法高出一个数量级。**
- **需要说明的是，尽管本章给出几种行之有效的截交算法，但不代表这些算法就是最优的。模型截交问题本身仍是一个开放的问题，期待同学们能够给出更加高效的算法。**





思考题

1. 在扫描平面法截交优化中，如何进一步提高从扫描平面维护的三角面片序列中删除面片的效率？
2. 尝试实现一种时间复杂度接近线性的排序算法。
3. 如何进一步提升代码5-2中intersect函数的执行效率。
4. 如果采用非均匀层高切片方法，如何确定每层层高以保证打印误差在可控范围内？
5. 简述扫描平面法和层高匹配法的基本原理。
6. 针对一个具体的STL模型，比较IntersectStl_Match类中matchFacetZs_brutal函数和matchFacetZs_bisection函数的执行效率。
7. 除扫描平面法和层高匹配法外，尝试设计一种新的STL模型快速截交算法。